

Shomer.AI — תכנית הכנה למפגש 5

מועד מתוכנן: TBD (~14-7 ימים אחרי מפגש 4) מנחה: ד"ר יורם סגל מציגה: אלונה מטרות המפגש: הוכחת DictaBERT-base עם $\text{macro-F1} \geq 0.78$ על SinaLab + הוכחת שרת מודולרי עם `classify end-to-end/` דרך ה-SDK CLI. מקורות: [design/tasks_index.json](#) · [design/review.md](#) · [design/](#) · [PRD.md](#) עדכון: 2026-06-03

1 · תקציר מנהלים

מפגש 4 אישר את הארכיטקטורה ואת ה-PRD. מפגש 5 הוא המעבר מתכנון לבנייה. שלוש מסלולי עבודה (Tracks) רצים במקביל, ובסוף יש משוואה אחת:

מפגש 5 מצליח $\Leftrightarrow \text{macro-F1} \geq 0.78$ על `shomer-cli demo` + `SinaLab test split` עובד מקצה לקצה דרך הארכיטקטורה המודולרית.

ה-(`docs/design/tasks_index.json`) Backlog מכיל 144 משימות מקודדות. מפגש 5 צורך חלק מהן — בעיקר `IF-01-*` (כל Protocol), `SDK`, `AUDIT-*`, `CLASSIFIER-*`, ו-`SDK-CLI-01/02`. השאר נכנסים במפגש 6 (Context Agent) ו-7 (Triage + Alerts מלאים).

2 · תוצרים נדרשים למפגש 5

תחום	תוצר	מקור	סטטוס
אקדמי	outputs/dictabert-offensive/ checkpoint macro_f1 ≥ 0.78 על SinaLab test split	classifier LLD \$7.2 + PRD \$8.1	דורש  בנייה
אקדמי	training/evaluate.py report (calibration) עם per-class F1, ECE < 0.10	classifier LLD \$7.3	דורש  בנייה
הנדסי	classify end-to-end/ (seams, in-process composition) על ארכיטקטורה מודולרית (Protocol)	server LLD \$6a + audit_log LLD	דורש  בנייה
הנדסי	audit.db עם trace_id מכל בקשה, retention sweep פעיל	audit_log LLD \$5, \$9	דורש  בנייה
הנדסי	Triage עם 6-row normalization (G-03 fix); prob_offensive polarity matrix עובר	trriage LLD \$3	דורש  בנייה
דמו	shomer-cli demo ו- python scripts/dev_client.py demo שניהם עוברים מול שרת רץ	sdk LLD \$3.5 + server LLD \$9a	דורש  בנייה
תזה	docs/implementation-summary-2026-06-XX.md (אופציונלי, דרך (hebrew-ai-project-manager	מנהג מהמפגש הקודם	 אופציונלי

3 · Pre-flight checklist (חובה לפני כתיבת קוד ראשונה)

משך: 1~ שעה.

1. שחזור venv של השרת (הנתיבים נשברו בהגירה ב-2026-05-23):
powershell cd C:\AIDevelopmentCourse\Shomer.AI\server Remove-Item -Recurse -Force .venv -
ErrorAction SilentlyContinue python -m venv .venv ; .\venv\Scripts\Activate.ps1
pip install -r requirements.txt
2. Smoke-test ה- ollama list — stand-in מראה offensive-hebrew:v1 ; uvicorn
server.app.main:app --reload עולה; POST /classify עם טקסט עברי מחזיר 200. זה ה-rollback
.baseline
3. git add — Git baseline כל מה שלא ב- v0.4-design-frozen ; tag tracking; commit; tag
/ docs/design/, plan-docs/, CLAUDE.md, prompts אינם committed.
4. WSL2 + CUDA verification — בתוך WSL2: python -c "import torch;
print(torch.cuda.is_available())" ⇒ True . RTX 5080 Blackwell דורש +CUDA 12.8 ו-
sm_120-aware wheels . אם זה לא עובד — Track A לא יכול להתחיל.
5. Android Studio — לפתוח את /android_client מהנתיב החדש; File → Invalidate Caches &
Restart אם Gradle sync נכשל בגלל caches ישנים.

⚠️ **שלב 4 הוא ה-Single Point of Failure של הלוח זמנים.** אם CUDA לא עובד ב-WSL2 — חצי יום עד יומיים לפתרון לפני שטראק A יכול להתחיל. כדאי לאמת בתחילת היום הראשון.

4 · Track A — אימון DictaBERT (תוצר אקדמי, WSL2 / RTX 5080)

משך משוער: 3–4 ימים. **קריטי:** כשל כאן = כשל במפגש 5.

#	משימה	קישור LLD	תוצר
A1	training/prepare_data.py — טעינת SinaLab Offensive-Hebrew → train/val/test 80/10/10 JSONL	classifier LLD §5.1	· data/train.jsonl · data/validation.jsonl · data/test.jsonl
A2	כתיבה של training/train_dictabert.py (חדש — train_lora.py הקיים הוא ל-generative models, לא לא-nkoder)	classifier LLD §3.3 hyperparam table	סקריפט אימון
A3	הרצת אימון ב-WSL2: AutoModelForSequenceClassification, BF16, AdamW + cosine, 5 epochs	classifier LLD §3.3	outputs/dictabert-offensive/checkpoint
A4	training/calibrate.py — isotonic validation split על regression	classifier LLD §3.4 + §7.3	server/models/calibrator.pkl
A5	training/evaluate.py — בדיקה macro_f1 ≥ 0.78 + per-class report + ECE < 0.10	classifier LLD §7.2 + §7.3	דו"ח Markdown + assert גלובלי
A6	העתקת checkpoint ל-/server/models/dictabert-offensive	classifier LLD §9.2	מודל זמין לשרת

אם chain **F1 < 0.78** — fallback מ-2. DictaBERT-large (~330M params; ~1GB VRAM) PRD §12: 1. DictaLM 2.0 (QLoRA, חוזרים ל-Ollama)
עלות זמן: שבוע נוסף בכל פעם. **לכן Track A מתחיל ביום 1 ולא מתעכב.**

5 · Track B — Refactor שרת לארכיטקטורה מודולרית (Sprint 1)

משך משוער: 5–7 ימים. רץ במקביל ל-Track A מיום 1 (ה-stand-in נשאר עובד עד שה-DictaBERT נוחת).

שלב 10 — B.1 Protocol Definitions (יום 1–2)

כל המודולים מקבלים את ה-Port שלהם לפני שכל adapter נכתב. סדר מומלץ:

1. AUDIT-IF-01 — חוסם את ALERTS-DB-01, SERVER-LIFESPAN-01, CTX-TOOLS-01
2. CLASSIFIER-IF-01 — חוסם את TRIAGE-IF-01
3. OCR-IF-01, GK-IF-01, TRIAGE-IF-01, CTX-IF-01, ALERTS-IF-01
4. SDK-IF-01, SERVER-IF-01, ANDROID-IF-01

שלב B.2 — Audit Log (יום 2–3)

משימה	תוצר
AUDIT-SCHEMA-01	5 טבלאות (classifications, agent_traces, alerts, conversations, gold_set_metadata) עם WAL mode SQLite
AUDIT-CONVO-01	API read_conversation_history(child_id, last_n) ל-Context Agent (מפגש 7)
AUDIT-RETENTION-01	RetentionSweeper רקעי, 7 ימים rolling
AUDIT-CT-01	Contract test parametrized על כל adapter

שלב B.3 — Classifier module (יום 3–5, תלוי ב-Track A)

משימה	תוצר	תלות
CLASSIFIER-OLLAMA-01	עטיפת classifier.py הקיים כ- adapter OllamaDictaBertClassifier (stand-in נשאר עובד)	CLASSIFIER-IF-01
CLASSIFIER-HF-01	adapter HuggingFaceClassifier, טוען את ה-checkpoint מ-Track A	CLASSIFIER-IF-01, Track A6
CLASSIFIER-CALIBRATION-01	טעינת hooks, calibrator.pkl ל-pipeline	Track A4
CLASSIFIER-CT-01	Contract test שני adapters עוברים אותו	CLASSIFIER-IF-01

החלפה מ-stand-in ל-DictaBERT אמיתי = שינוי שורה אחת ב-.env (CLASSIFIER_MODEL_VERSION=v1.1 - dictabert).

שלב B.4 — Triage + Alerts (יום 5–6)

משימה	תוצר
TRIAGE-RULE-01	RuleBasedTriage עם <code>prob_offensive normalization</code> Step A (תיקון G-03)
TRIAGE-CT-01	trriage LLD §3 מ- row polarity matrix — חובה לעבור
ALERTS-FCM-01	FCM integration + idempotency key
ALERTS-DB-01	רישום ל- <code>alerts table</code> דרך <code>AuditStore</code>

Context Agent נשאר לא-מחובר במפגש 5 ; ה-pipeline בלי CA מקצה לקצה הוא בעצם ה-A/B baseline שמודדים מולו במפגש 8.

שלב B.5 — Composition root + dev tools (יום 6–7)

משימה	תוצר
SERVER-LIFESPAN-01	<code>main.py lifespan</code> () בונה את הגרף ; כל החלפת <code>adapter</code> = שינוי שורה
SERVER-CLASSIFY-01	<code>/classify</code> endpoint חדש על ה-pipeline המודולרי
SERVER-HEALTH-01	<code>health/</code> עם <code>health rollup</code> מכל מודול , <code>AuditStore.health()</code> , <code>TextClassifier.is_alive()</code> , וכו')
SERVER-DEV-CLI-01	Python CLI — <code>scripts/dev_client.py</code> מהיר לבדיקות
SERVER-DEV-INSPECT-01	SQLite inspector — <code>scripts/inspect_audit.py</code>

Track C — SDK + Terminal CLI · 6

משך משוער: 4–5 ימים. רץ במקביל ל-Track B מיום 1 (תלוי רק בקיום של stand-in — endpoints מספיק להתחיל).

שלב C.1 — ספרייה (יום 1–3)

- SDK-IF-01 — `ShomerResult` sealed + `ShomerError` hierarchy + `ShomerEndpoint<I, O>` Protocol
- SDK-CFG-01 — `SdkConfig` data class (`baseUrl` , `timeouts` , `retries` , `sdkVersion`)
- SDK-MODELS-01 — מירור Kotlin data classes ל- `server/app/schemas.py`
- SDK-HTTP-01 — `OkHttp` + `Retrofit` + `retry interceptor` (1s/2s/4s) + `trace_id` propagation
- SDK-ENDPOINTS-01 — 4 endpoints (`classify` , `classify-image` , `health` , `info`)
- SDK-CLIENT-01 — `ShomerClient` public API

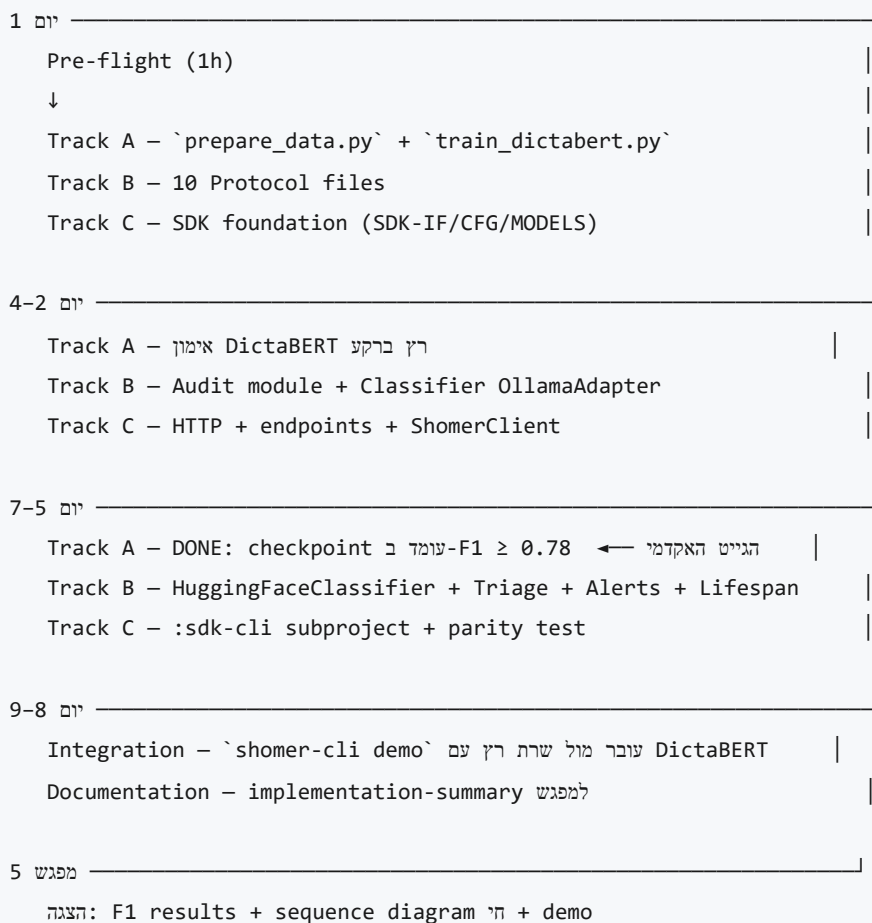
שלב C.2 — Terminal CLI (יום 3-4)

9. SDK-CLI-01 — Gradle subproject :sdk-cli (clikt + moshi); fat-jar .
8. SDK-CLI-02 — 5 subcommands: `classify` / `classify-image` / `health` / `info` / `demo` .
ה- `demo` רץ על `golden_inputs.jsonl` עם 8-10 דוגמאות עבריות.

שלב C.3 — Contract tests (יום 4-5)

9. SDK-CT-01 — ShomerApi Protocol contract test .
10. `shomer-cli demo` — **Parity test** ו- `python scripts/dev_client.py demo` חייבים להחזיר פלט מבני זהה. זה ה- cross-language wire-protocol contract test (גם סוגר חלק מ-G-05).
- (batch mode) SDK-CLI-03 נדחה ל-מפגש 8 (gold-set runs).

7 · Critical path & תזמון



Realistic readiness: ~10 ימי עבודה מאדם אחד. מפגש 5 בעוד שבועיים מ-Pre-flight.

חלק 1 — אימות הגייט האקדמי (5 דקות)

- `training/evaluate.py` output: per-class F1 + macro-F1 + ECE
- Confusion matrix על test split
- אם $F1 \geq 0.78$ —  . אם לא — מציגה את ה-fallback chain שכבר רץ ברקע.

חלק 2 — Live demo (5 דקות)

- `uvicorn server.app.main:app` עולה
- `java -jar shomer-cli.jar demo --server http://localhost:8000` — golden set
- `python scripts/inspect_audit.py tail -n 5` — מראה ש-trace_id נכנס לכל שורה ב-audit.db

חלק 3 — הסבר ארכיטקטוני (5 דקות)

- מציגה את **Composition root pattern** מ-`server/app/main.py` : שינוי שורה אחת בין stand-in ל-DictaBERT
- מסבירה איך ה-**normalization** **prob_offensive** ב-triage מונע את ה-FP-storm שהיינו מקבלים אחרת (התיקון של G-03)
- מצביעה על `tests/contracts/test_triage_engine_contract.py` עם 6-row polarity matrix

חלק 4 — Roadmap למפגש 6 (5 דקות)

- מפגש 6 = סינתזת שיחות עבריות מתויגות + Context Agent pilot
 - תלוי בסגירת Q4 + Q1 (MoSCoW) Open Q1 (retention) —  נסגר ב-audit_log LLD + Q6 (single-child)
 - ה-AUDIT-CONVO-01 כבר במקום → `read_conversation_history` tool של ה-Context Agent ראדי
-

9 · סיכונים ומיטיגציות

סיכון	סבירות	חומרה	מיטיגציה
CUDA / sm_120 wheels לא עובדים ב-WSL2	בינונית	גבוהה	אימות ב-Pre-flight ביום 1 ; fallback: docker ; NVIDIA CUDA image
DictaBERT-base לא חוצה F1 0.78	בינונית	בינונית	DictaBERT-large או DictaLM-2.0 (PRD §12) ; DictaLM-2.0 ; +שבוע
Tesseract גרוע על צילומי מסך אמיתיים	גבוהה	נמוכה במפגש 5	OCR נדחה ל-מפגש 6 ; לא חוסם את מפגש 5
Migration של /server/app לארכיטקטורה החדשה שובר את ה-stand-in	בינונית	גבוהה	כל קומיט נשמר ; feature-flag-able ; OllamaClassifier adapter נשאר זמין במקביל
Gradle setup ב-Kotlin SDK איטי בפעם הראשונה	גבוהה	נמוכה	יום שלם של buffer בטראק C
Inter-track integration קופץ ביום 8	בינונית	בינונית	יום 8-9 buffer מוקדש לזה

10 · שאלות פתוחות לפני מפגש 5

מתוך 7 השאלות הפתוחות ב- docs/open_questions.md , אלה שצריכות סגירה לפני מפגש 5 (השאר נשארות לאחר):

#	שאלה	סטטוס	פעולה
Q1	MoSCoW סופי לפיצ'רים	פתוח	סשן ייעודי שבוע לפני מפגש 5
Q4	שמירת היסטוריית שיחות	נסגר ב-§5, §9 audit_log/design.md (7 ימים SQLite WAL, cron rolling)	לעדכן open_questions.md
Q6	Single-child / Family	פתוח	סשן ייעודי לפני מפגש 5

G-04 (port-naming drift): בחירה בין TriageRouter / TriageEngine (וכו') + ~30 grep-replace. רצוי לפני B.1 כי הקוד שייכתב יקבע את השם הקנוני.

(**G-05 (Android ClassificationSource throws ↔ SDK ShomerApi returns ShomerResult<T>)** : רצוי לפני C.1 ; אם לא — נסגר בעצם דרך הפריטה ב-C.3.

11 · מה לעשות מיד אחרי המפגש

- 1. אם demo + $F1 \geq 0.78$ עובד $\rightarrow$ tag v0.5-meeting-5-pass ; pushing לרפו הציבורי.
- 2. אם F1 לא חצה $\rightarrow$ tag v0.5-rc1 ; מתחילים מיד את ה-fallback chain (DictaBERT-large).
- 3. תוך 3–5 ימים : סשן Open Q1 (MoSCoW) + Q6 (single-child) — חוסמים את מפגש 6.
- 4. שבוע אחרי : מתחילים מפגש 6 (סינתזת שיחות + Context Agent pilot).

12 · תוצרים מקומיים שיוגשו (קישורים)

תוצר	נתיב מקומי	סטטוס
Training pipeline	training/{prepare_data,train_dictabert,calibrate,evaluate}.py	Track A
Checkpoint	server/models/dictabert-offensive/ + calibrator.pkl	Track A
Evaluation report	training/reports/eval_dictabert_YYYY-MM-DD.md	Track A
שרת מודולרי	server/app/{audit_log,classifier,triage>alerts}/ packages	Track B
audit.db schema migration	server/app/audit_log/migrations/001_init.sql	Track B
SDK Kotlin	server/sdk/kotlin/ (Gradle module)	Track C
SDK-CLI	server/sdk/kotlin-cli/ (Gradle subproject, fat-jar)	Track C
Dev tools	scripts/{dev_client,inspect_audit,load_test}.py	Track B
תיק זה (+ Hebrew (PDF	docs/meeting5_prep.md · docs/meeting5_prep.pdf	ייוצר אחרי כתיבה